

BANKING PORTAL

Java and Spring Boot Based Banking Management System

Submitted by: Aryan Sharma

Registration Number: 25BAI11160

Programme: B.Tech – Computer Science and Engineering (AIML)

University: VIT Bhopal University

1. Introduction

Banking Portal is a backend application I built using Java and Spring Boot. The idea is simple: take the everyday things a person does at a bank (checking a balance, depositing, withdrawing, sending money to someone) and expose them as REST APIs.

A new user registers and logs in. Once the login succeeds, the server hands back a JWT, and that token unlocks everything else: viewing the account, setting or changing a PIN, depositing, withdrawing, transferring funds, and pulling up past transactions. User profiles, account records, and transaction entries are all stored in MySQL.

Since the project is backend-only, there is no interface to click through. I used Postman as the client, which made it easy to send requests, inspect responses, and confirm each endpoint behaved as expected.

1.1 Objectives

1. Build a working banking backend with Java and Spring Boot.
2. Expose the common banking operations as REST APIs.
3. Authenticate users with JWT instead of repeated credential checks.
4. Persist user, account, and transaction data in MySQL.
5. Support deposits, withdrawals, and transfers between accounts.
6. Keep a record of every transaction.
7. Handle failures through proper exception handling.
8. Test every endpoint in Postman, including the cases that should fail.

2. Problem Statement

Anyone using a bank does the same handful of things repeatedly: checking the balance, moving money in or out, sending it to someone, and going back through old transactions.

Doing this digitally sounds straightforward until you look at the details. The system has to be certain who is making the request, refuse anyone who is not authorised, and make sure that when money moves, the numbers on both sides end up right every time. A half-completed transfer is worse than no transfer at all.

This project addresses that with a backend where authenticated users carry out banking operations through APIs, built on Java, Spring Boot, MySQL, and JWT.

3. Functional Requirements

Registration and login. A user signs up with their name, email, phone number, address, and password. Nothing is saved until the input passes validation. On a successful login, the server issues a token that stands in for their identity on every request after that.

Authentication. Every protected request has to carry the token in its header:

```
Authorization: Bearer <JWT Token>
```

No token, or one that does not hold up, means no access.

Account and PIN management. A logged-in user can view their account details and balance, and can create, verify, or change a PIN. The PIN is a second check before an operation touches the balance.

Deposit. The system confirms the user, reads the amount, adds it to the balance, and writes a transaction record.

Withdrawal. The same in reverse, with one extra step: the balance is checked first. If there is not enough money, the request is refused and nothing is deducted.

Fund transfer. The system confirms the sender, confirms the receiving account exists, checks the sender can cover the amount, deducts from one side, credits the other, and records the transaction. The deduction and the credit belong together: either both happen or neither does.

Transaction history. A user can list everything they have done on the account, with the type, amount, timestamp, counterparty, and status of each entry.

Error handling. The application has to cope with the predictable failures: unknown user, unknown account, unauthorised request, invalid input, and insufficient balance.

4. Non-Functional Requirements

Requirement	What it means here
Security	JWT keeps unauthorised requests away from protected endpoints
Performance	Requests handled efficiently, with no noticeable delay
Reliability	Banking operations give the correct result consistently
Scalability	Room to add more users and features without a rebuild
Maintainability	Layered code, so a change in one place does not ripple outward
Usability	Predictable request formats and error messages that explain the problem
Data integrity	Balances stay consistent during deposits, withdrawals, and transfers

5. System Architecture

Banking Portal follows a layered structure, with each layer handling one kind of concern.

```
USER / CLIENT  --> Controller  --> Service  --> Repository  --> MySQL
(Postman)      (REST API)    (business  (data access)
                  logic)
```

Controller layer. Controllers sit at the edge. They take the incoming HTTP request, hand it off, and send back the response, and nothing more.

```
POST /api/users/register      POST /api/account/withdraw
POST /api/users/login         POST /api/account/fund-transfer
POST /api/account/deposit     GET  /api/account/transactions
```

Service layer. Where the real work happens. In a transfer, the service checks whether the sender has enough money, updates both balances, and saves the transaction record. The controller knows none of that.

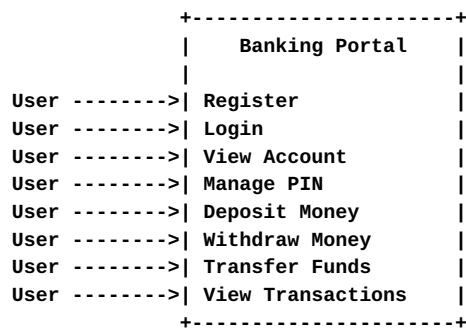
Repository layer. Talks to MySQL and nothing else: saving records, looking up users and accounts, updating balances, fetching transactions.

Database layer. MySQL holds the three entities the application needs to remember: User, Account, and Transaction.

6. Design Diagrams

6.1 Use Case Diagram

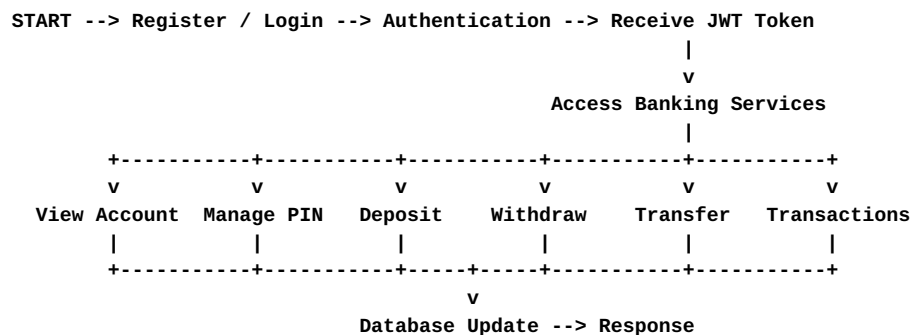
There is one actor, the **User**, who can do the following:



Everything past Register and Login sits behind authentication, so those two entry points gate the rest.

For the final submission, replace this sketch with a proper UML use case diagram.

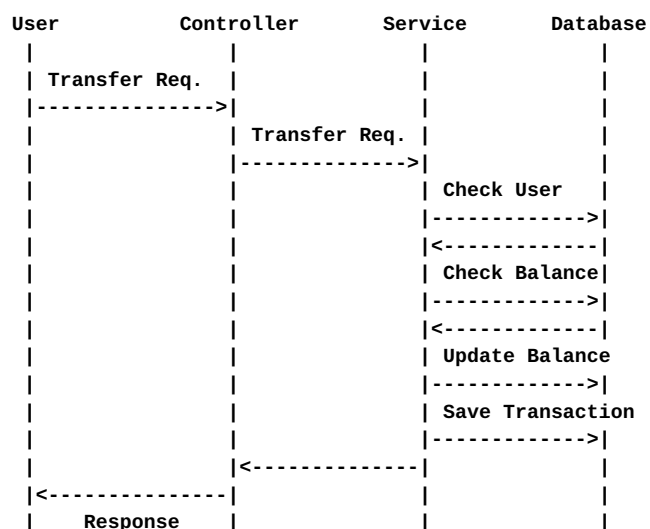
6.2 Workflow



The token is the hinge in the middle. Nothing below it is reachable without one.

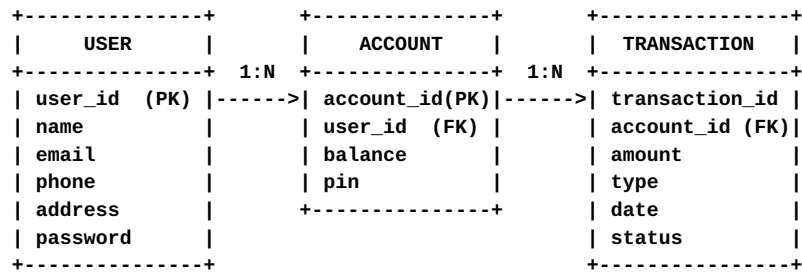
6.3 Sequence Diagram: Fund Transfer

Of all the operations, the transfer touches the most moving parts, so it is worth tracing in full.



The user sends the request with a target account and amount. The controller passes it down, and the service verifies both accounts, checks the sender's balance, updates both sides, and saves a transaction record before the response travels back out. If any check in the middle fails, the process stops there and an error goes back instead, with no partial update.

6.4 ER Diagram



A user owns an account, and an account collects many transactions over time. The final diagram should mirror the actual tables and foreign keys in the project.

6.5 Class / Component Structure

Controllers	UserController, AccountController
Services	UserService, AccountService
Repositories	UserRepository, AccountRepository, TransactionRepository
Entities	User, Account, Transaction
Requests	LoginRequest, RegisterRequest, AccountRequest, TransferRequest

Each request enters through a controller, is processed by the matching service, and reaches the database through a repository.

7. Design Decisions

Java and Spring Boot. Java's object-oriented model maps cleanly onto what this project deals with: a user, an account, and a transaction are all obvious classes. Spring Boot then removes most of the setup that would otherwise come before writing any banking logic, since configuration, REST endpoints, dependency injection, security, and database access all come ready to use.

MySQL. Banking data is highly structured and the relationships between users, accounts, and transactions matter. A relational database enforces those relationships rather than leaving it to application code.

REST APIs. Using standard HTTP methods keeps the backend independent of whatever ends up calling it, so a web frontend, an Android app, or Postman can all talk to the same server.

JWT. The user logs in once and carries a token from then on. There is no session to store on the server, and credentials are not resent with every request.

Layered architecture. Each layer has one job. When something breaks, I know which layer to open, and changing the data access logic does not force me to touch the endpoints.

8. Implementation

The code is split into three modules. The **user module** handles registration, login, and user details. The **account module** covers viewing the account, checking the balance, and creating, verifying, or updating the PIN. The **transaction module** handles deposits, withdrawals, transfers, and transaction history.

8.1 Authentication Flow

```
Login --> Credentials Verified --> JWT Generated --> Token Sent to User
                                     |
                                     v
Access Granted <-- Token Verified <-- Token Added to Protected Request
```

Once issued, the token is checked on every protected request before the controller ever sees it.

8.2 Exception Handling

Rather than scattering error handling through the controllers, the application catches problems in one place and turns them into proper HTTP responses.

Exception	When it is thrown	Status
NotFoundException	The requested user or account does not exist	404 Not Found
UnauthorizedException	The token is missing, invalid, or the user has no right to the resource	401 Unauthorized
InsufficientBalanceException	The account cannot cover the transaction	400 Bad Request

9. Testing

Everything was tested through Postman, both the paths that should work and the ones that should fail. Each endpoint was called on its own to confirm it did its job. The protected endpoints were hit three ways, with a valid token, a tampered one, and no token at all, to confirm only the first got through. Incomplete and malformed input was sent deliberately to check the system rejects it with a useful message instead of failing deeper in the stack. Deposits, withdrawals, and transfers were run while watching the balances on both sides.

Test Case	Expected Result
Valid login	Successful login
Invalid login	Unauthorized response
Valid deposit	Balance increases
Valid withdrawal	Balance decreases
Insufficient balance	Error response
Invalid account	Not Found response
Missing token	Unauthorized response
Valid fund transfer	Transfer successful

9.1 Results

Screenshots from Postman and MySQL Workbench are attached, covering registration, login and the returned JWT, account details, a deposit, a withdrawal, a fund transfer, transaction history, and the stored database records.

10. Challenges Faced

Getting JWT working properly was the first real hurdle. Generating a token is easy enough, but validating it on every request and making sure the right endpoints were protected while login and registration stayed open took considerably more effort.

Wiring Spring Boot to MySQL was the second. Mapping the relationships between users, accounts, and transactions meant getting the annotations and configuration exactly right, and small mistakes here surfaced as confusing errors much later.

Transfers were the trickiest operation, since two accounts change at once and a transaction record has to be written alongside them. Everything has to succeed together or not at all.

Beyond that, it was tempting to return a generic failure everywhere, so working out which situation deserved which status code took a few passes. Testing in Postman also meant tracking headers, tokens, bodies, and expected status codes across every endpoint, and remembering to test the cases that were meant to fail.

11. Learnings

On the technical side, the project covered Spring Boot, REST API design, MySQL, JWT and Spring Security, Maven, Postman, and Git.

The bigger takeaway was seeing how the pieces connect. A request follows a clear path from controller to service to repository to database and back out again. Once that path was clear in my head, adding a feature stopped feeling like guesswork. I also came away with a much better sense of why authentication and error handling get designed up front rather than bolted on at the end.

12. Future Enhancements

The most useful additions would be email notifications for logins and transactions, generated monthly bank statements, and a dashboard showing the balance, recent activity, and spending charts. Transaction history returns everything at once right now, so pagination would keep it fast as accounts build up entries.

Security has the most room left: stronger password hashing, proper token expiry and refresh handling, multi-factor authentication, and extra verification on high-value transactions. Finally, a React or Android frontend would let someone actually use the system instead of calling the APIs by hand.

13. References

1. **Oracle Java Documentation** – Java language features and APIs.
2. **Spring Boot Documentation** – Application setup and configuration.
3. **Spring Security Documentation** – Authentication and authorization.
4. **MySQL Documentation** – Database management and SQL.
5. **JWT Documentation** – Token-based authentication.
6. **Postman Documentation** – API development and testing.
7. **GitHub Documentation** – Repositories and version control.

Conclusion

Banking Portal is a working backend for a banking system, built with Java and Spring Boot. It handles registration and login, account and PIN management, deposits, withdrawals, transfers, and transaction history, with JWT locking down the protected endpoints and MySQL giving the data a structure that matches how the entities actually relate.

More than the feature list, what I take from this project is the experience of building a backend properly: designing REST APIs, connecting them to a database, handling authentication, dealing with errors, testing everything, and keeping the code organised well enough that I could still work with it weeks later.